

PHP Developer

BriBooks Mini Book Writing Platform

Overview

Build a simplified version of the BriBooks book creation and publishing platform. The system should allow authors to create books, manage versions, upload manuscripts, submit books for review, and publish approved books.

This assignment is designed to evaluate your ability to build scalable backend systems, design clean APIs, implement workflows, and write maintainable PHP code.

Expectation

You are not expected to build a production-ready application. Focus on clean architecture, code quality, and thoughtful implementation.

Technical Requirements

Mandatory

- PHP 8.3+
- MySQL 8+
- REST APIs
- JWT Authentication
- PHPUnit Tests

Preferred Architecture

Model View Controllers
Services
Request Validation
Events

Framework choice is up to you:

Laravel
CodeIgniter
Custom PHP MVC Framework

Business Requirements

1. User Authentication

Roles

- Author
- Reviewer
- Admin

APIs

- POST /api/register
- POST /api/login
- GET /api/profile
- POST /api/logout

Expectations

- JWT-based authentication
- Role-based authorization
- Secure password hashing

2. Book Management

Authors can create and manage books.

APIs

POST /api/books

GET /api/books

GET /api/books/{id}

PUT /api/books/{id}

DELETE /api/books/{id}

Rules

- Authors can only manage their own books.
- Published books become read-only.

3. Book Version Control

Every significant update should create a new version of the book.

APIs

POST /api/books/{id}/versions
GET /api/books/{id}/versions
GET /api/books/{id}/versions/{versionId}

Requirements

Store:

Book Metadata
Chapters
Pages

inside a version snapshot.

Evaluation

We are interested in your approach to:

- Version storage
- Snapshot design
- Rollback capability
- Scalability

4. Chapter Management

Books contain chapters.

APIs

POST /api/books/{id}/chapters
PUT /api/chapters/{id}
DELETE /api/chapters/{id}
GET /api/books/{id}/chapters

5. Page Management

Each chapter contains pages.

APIs

POST /api/chapters/{id}/pages
PUT /api/pages/{id}
DELETE /api/pages/{id}
GET /api/chapters/{id}/pages

6. Document Upload & Conversion

Allow users to upload manuscript files.

Supported Formats

.doc
.docx

API

POST /api/books/{id}/upload

Requirement

Convert uploaded documents into multiple HTML pages.

7. Moderation System

When a book is submitted, perform content moderation.

API

POST /api/books/{id}/submit

Checks

At minimum:

Profanity Detection
Restricted Words Detection

8. Review & Publishing Workflow

Workflow

Draft —> Submitted —> Under Review —> Approved —> Published

APIs

POST /api/books/{id}/submit
POST /api/books/{id}/approve
POST /api/books/{id}/reject
POST /api/books/{id}/publish

Rules

Only Authors can Submit
Only Reviewers can Approve or Reject
Only Admins can Publish
Published Books are Read-Only

9. Author Dashboard

API

GET /api/dashboard

Unit Testing

Provide PHPUnit tests for critical functionality.

Examples:

Authentication
Book Creation
Version Creation
Workflow Transitions
Moderation
Authorization Rules

Bonus Features (Optional)

These are not required but will significantly improve your evaluation score.

Redis Integration

Examples:

Dashboard Caching
Book Statistics
Frequently Accessed Books

Queue System

Move heavy operations into background jobs.

Examples:

Document Conversion
Moderation
Email Notifications
Publishing Tasks

OpenAI Integration

Generate:

Book Summary
Book Tags
Reading Level
Writing Suggestions

Example API:

POST /api/books/{id}/ai-summary

Event-Driven Architecture

Implement events such as:

BookCreated
BookVersionCreated
BookSubmitted
ModerationPassed
BookApproved
BookPublished

Examples:

BookSubmitted—>ModerationListener—>BookPublished—>NotificationListener—
>BookCreated—>AnalyticsListener

Submission Requirements

Provide:

Git Repository
README.md
Database Migration Files
API Documentation
Sample Environment Configuration

README should include:

Installation Steps

How to Run Tests

API Usage Instructions

Architecture Decisions

Assumptions Made

What We Are Looking For

We are evaluating:

- Clean and maintainable code
- Thoughtful architecture decisions
- Proper database design
- API consistency
- Security awareness
- Testing discipline
- Scalability considerations
- Problem-solving approach

Focus on demonstrating engineering quality rather than implementing every possible feature. Clearly document any assumptions or trade-offs made during development.